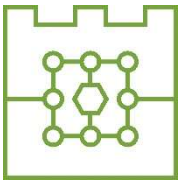


Politechnika Krakowska
im. Tadeusza Kościuszki

Wydział Informatyki i Matematyki



Sprawozdanie

Zaawansowane technologie baz danych

Porównanie wybranych relacyjnych i nierelacyjnych baz danych na podstawie albumu gwiezdnego

Michał Wilk 147017

Wiktor Zmiendak 142706

Kraków 2026

1. Wstęp

W niniejszej pracy przyjęto podejście porównawcze, którego celem jest ocena zachowania dwóch relacyjnych i dwóch nierelacyjnych systemów baz danych w warunkach rosnącego obciążenia. Praca koncentruje się na pytaniu, w jaki sposób model danych oraz mechanizmy indeksowania wpływają na wydajność operacji CRUD i jakość obsługi zapytań analitycznych. Analiza została przeprowadzona na wspólnym zbiorze danych z domeny astronomii oraz w ujednoliconym środowisku kontenerowym, co zapewnia porównywalność wyników.

W badaniu przyjęto zestaw scenariuszy testowych obejmujących operacje tworzenia, odczytu, aktualizacji i usuwania danych dla trzech skal wolumenu rekordów. Dla każdego scenariusza wykonano parę prób, a wyniki końcowe opracowano na podstawie otrzymanych wartości, dodatkowo porównując pomiary przed i po zastosowaniu indeksów. Ocenie poddano nie tylko czasy wykonania operacji, ale również aspekty bezpieczeństwa, niezawodności, integracji i skalowalności badanych systemów.

Zakres pracy obejmuje analizę środowiska lokalnego i nie uwzględnia pełnych testów rozproszonych w infrastrukturze chmurowej. Tak określone granice badania pozwalają skupić się na rzetelnym porównaniu cech silników bazodanowych w kontrolowanych warunkach eksperymentalnych.

Wybrane SZBD obejmują:

- PostgreSQL 15
- MySQL 8.0
- MongoDB 7
- Neo4j 5

2. PostgreSQL

2.1 Charakterystyka

PostgreSQL to zaawansowany, obiektowo-relacyjny system zarządzania bazami danych o otwartym kodzie źródłowym, znany z wysokiego poziomu protokołowania, zaawansowanych funkcji SQL oraz doskonałej zgodności ze standardami. Cechuje się wysoką współbieżnością dzięki wsparciu wielowersyjności (MVCC). Posiada natywne wsparcie dla JSON oraz JSONB.

2.2 Bezpieczeństwo

W kontekście bezpieczeństwa PostgreSQL oferuje wielowarstwowe mechanizmy kontroli dostępu. System umożliwia stosowanie nowoczesnych metod uwierzytelniania, w tym SCRAM-SHA-256, a także szczegółowe zarządzanie rolami i uprawnieniami na poziomie schematów, tabel, widoków czy funkcji. Komunikacja klient-serwer może być zabezpieczona protokołem TLS, a dane mogą być dodatkowo chronione przez rozszerzenia kryptograficzne. Ważną cechą jest również rozbudowane logowanie zdarzeń, które wspiera audyt oraz analizę incydentów bezpieczeństwa.

2.3 Awaryjność

Od strony niezawodności PostgreSQL opiera się na mechanizmie ACID dzięki czemu gwarantuje trwałość danych nawet w przypadku awarii. System obsługuje odtwarzanie punktowe (PITR), co ma istotne znaczenie przy pracy z dużymi wolumenami danych i wymaganiami ciągłości działania. W praktyce oznacza to możliwość odtworzenia bazy do konkretnego momentu w czasie, np. tuż przed błędną operacją użytkownika.

2.4 Integracje

PostgreSQL wyróżnia się także możliwościami integracji. Dzięki sterownikom ODBC, JDBC i natywnym bibliotekom dla popularnych języków programowania łatwo łączy się z aplikacjami webowymi, systemami backendowymi i narzędziami ETL. W praktyce często stosowany jest zarówno jako główna baza transakcyjna, jak i źródło danych dla hurtowni oraz platform raportowych.

2.5 Zalety

Największymi zaletami PostgreSQL są bardzo dobra zgodność ze standardem SQL, bogaty zestaw typów danych, wysoka jakość optymalizatora zapytań oraz dojrzałość mechanizmów transakcyjnych.. Mimo tych trudności PostgreSQL pozostaje jedną z najlepszych opcji wszędzie tam, gdzie kluczowe są spójność danych, przewidywalność działania i zaawansowane możliwości analityczne.

2.6 Wady

Za wyzwania można uznać większą złożoność konfiguracji, zwłaszcza przy budowaniu środowisk HA, oraz konieczność świadomego tuningu parametrów przy bardzo dużych obciążeniach.

3. MySQL

3.1 Charakterystyka

MySQL 8.0 jest jednym z najczęściej wybieranych relacyjnych systemów baz danych, szczególnie w środowiskach aplikacji webowych. Popularność tego silnika wynika z prostoty wdrożenia, szerokiego wsparcia narzędziowego oraz wieloletniej obecności w ekosystemie LAMP. W nowoczesnych instalacjach standardem jest wykorzystanie silnika InnoDB, który zapewnia transakcyjność ACID i obsługę kluczy obcych.

3.2 Bezpieczeństwo

W obszarze bezpieczeństwa MySQL istotnie rozwinął mechanizmy uwierzytelniania oraz zarządzania uprawnieniami. Domyślne metody logowania są bezpieczniejsze niż w starszych wydaniach, a model ról upraszcza administrację większymi zespołami użytkowników. W praktyce wdrożeniowej ważne jest także wsparcie dla szyfrowania połączeń oraz możliwość konfiguracji polityk haseł i limitów zasobów przypisanych do kont.

3.3 Awaryjność

Niezawodność MySQL zależy przede wszystkim od właściwej konfiguracji InnoDB oraz strategii replikacji. W codziennym użytkowaniu system bardzo dobrze sprawdza się w obciążeniach o dużej liczbie odczytów i średnio złożonych zapytaniach. Odtwarzanie po awarii

wspierają mechanizmy dzienników redo i undo oraz sprawdzony ekosystem narzędzi do backupu. Dla środowisk o podwyższonych wymaganiach dostępne są konfiguracje klastrowe oraz replikacja grupowa.

3.4 Integracje

MySQL ma również bardzo silną pozycję integracyjną. Jest wspierany przez praktycznie wszystkie popularne frameworki i języki programowania, a jego obecność w usługach chmurowych upraszcza zarządzanie infrastrukturą. Dla zespołów realizujących projekty biznesowe oznacza to krótszy czas wdrożenia i łatwiejsze utrzymanie. W scenariuszach typowo webowych, gdzie zapytania są przewidywalne, MySQL często oferuje bardzo dobry stosunek wydajności do kosztu administracji.

3.5 Zalety

- Prostota
- Popularność
- szeroka baza wiedzy i narzędzi

3.6 Wady

Ograniczeniem bywa mniejsza elastyczność względem najbardziej zaawansowanych konstrukcji SQL, a także konieczność ostrożnego projektowania pod kątem nietypowych zapytań analitycznych. Mimo to MySQL pozostaje solidnym i praktycznym wyborem dla dużej klasy aplikacji produkcyjnych.

4. MongoDB

4.1 Charakterystyka

MongoDB reprezentuje dokumentowy model danych, w którym informacje przechowywane są w formacie BSON. W odróżnieniu od podejścia relacyjnego struktura dokumentów może być elastyczna, co pozwala szybciej dostosowywać model danych do zmian wymagań biznesowych. Dzięki temu MongoDB jest często wybierane tam, gdzie występują dynamiczne schematy, rozproszone źródła danych lub intensywny rozwój produktu.

4.2 Bezpieczeństwo

Bezpieczeństwo w MongoDB obejmuje mechanizmy uwierzytelniania, autoryzacji i szyfrowania zarówno danych „w ruchu”, jak i „w spoczynku”. System wykorzystuje model RBAC, umożliwia pracę z certyfikatami oraz integrację z usługami zarządzania tożsamością.

4.3 Awaryjność

Niezawodność MongoDB opiera się na replikacji w postaci replica setów oraz automatycznym przełączaniu roli węzła głównego podczas awarii. W praktyce rozwiązanie to zapewnia wysoką dostępność usług i ogranicza ryzyko długich przestojów. MongoDB wspiera także skalowanie horyzontalne przez sharding, co pozwala efektywnie obsługiwać bardzo duże zbiory danych i wysoki ruch aplikacyjny.

4.4 Integracje

Silną stroną MongoDB jest ekosystem integracyjny. System oferuje sterowniki dla większości języków, łączy się z narzędziami strumieniowania danych, a w wersjach chmurowych dostarcza funkcje przyspieszające budowę nowoczesnych aplikacji. Dla zespołów deweloperskich oznacza to dużą swobodę projektową i krótszy cykl implementacji.

4.5 Zalety

Do zalet MongoDB należy:

- elastyczność modelu
- wygoda pracy z danymi zagnieżdżonymi
- dobra skalowalność

4.6 Wady

Ograniczenia pojawiają się przy złożonych relacjach i zapytaniach, które w modelu relacyjnym realizowane są naturalnie przez JOIN. Dodatkowo architektura dokumentowa wymaga ostrożnego projektowania pod kątem redundancji i spójności danych. Właściwie zaprojektowane środowisko MongoDB może jednak zapewnić bardzo wysoką wydajność i dużą odporność na wzrost obciążenia.

5. Neo4j

5.1 Charakterystyka

Neo4j jest bazą grafową, która koncentruje się na przechowywaniu i analizie relacji pomiędzy obiektami. W przeciwieństwie do modeli tabelarycznych i dokumentowych, tutaj relacje są elementem pierwszoplanowym, co znacząco przyspiesza zapytania wymagające wielokrotnego przechodzenia po powiązaniach. Z tego powodu Neo4j znajduje zastosowanie m.in. w systemach rekomendacyjnych, analizie sieci, wykrywaniu nadużyć czy budowie grafów wiedzy.

5.2 Bezpieczeństwo

W obszarze bezpieczeństwa Neo4j udostępnia mechanizmy zarządzania użytkownikami i rolami, a także wsparcie dla integracji z zewnętrznymi dostawcami tożsamości. Komunikacja sieciowa może być szyfrowana, a logowanie zapytań ułatwia audyt działań administracyjnych oraz analizę wykorzystania systemu.

5.3 Awaryjność

Niezawodność Neo4j w zastosowaniach produkcyjnych opiera się na mechanizmach klastrowych i replikacji. W konfiguracjach enterprise możliwe jest automatyczne przełączanie ról w klastrze oraz utrzymanie ciągłości dostępu do danych po awarii pojedynczego węzła. Dodatkowo system wspiera wykonywanie backupów i odtwarzanie danych, co jest kluczowe przy długoterminowej eksploatacji.

5.4 Integracje

Neo4j wyróżnia się też integracjami z narzędziami analitycznymi i ekosystemem data engineering. Język Cypher, mimo że wymaga innego sposobu myślenia niż SQL, pozwala bardzo czytelnie zapisywać zapytania o złożone wzorce relacyjne. W praktyce, gdy głównym problemem biznesowym jest analiza relacji, Neo4j potrafi dostarczyć wyniki szybciej i prościej niż klasyczne podejście relacyjne.

5.5 Zalety

Największym atutem systemu jest wydajność traversowania grafu oraz naturalność modelowania danych połączonych.

5.6 Wady

Ograniczeniem może być mniejsza popularność niż SQL, co przekłada się na mniejszą liczbę specjalistów i materiałów edukacyjnych. Mimo to, w scenariuszach grafowych Neo4j stanowi rozwiązanie bardzo skuteczne i często trudne do zastąpienia przez inne modele danych.

6. Gwiezdny album

Przygotowany zbiór danych został zaprojektowany w domenie astronomii, aby umożliwić rzetelne porównanie relacyjnych i nierelacyjnych modeli przechowywania informacji. Model obejmuje dziesięć głównych encji, które reprezentują zarówno obiekty astronomiczne, jak i elementy procesu obserwacyjnego. W praktyce oznacza to połączenie danych referencyjnych, hierarchicznych i operacyjnych w jednym spójnym środowisku testowym.

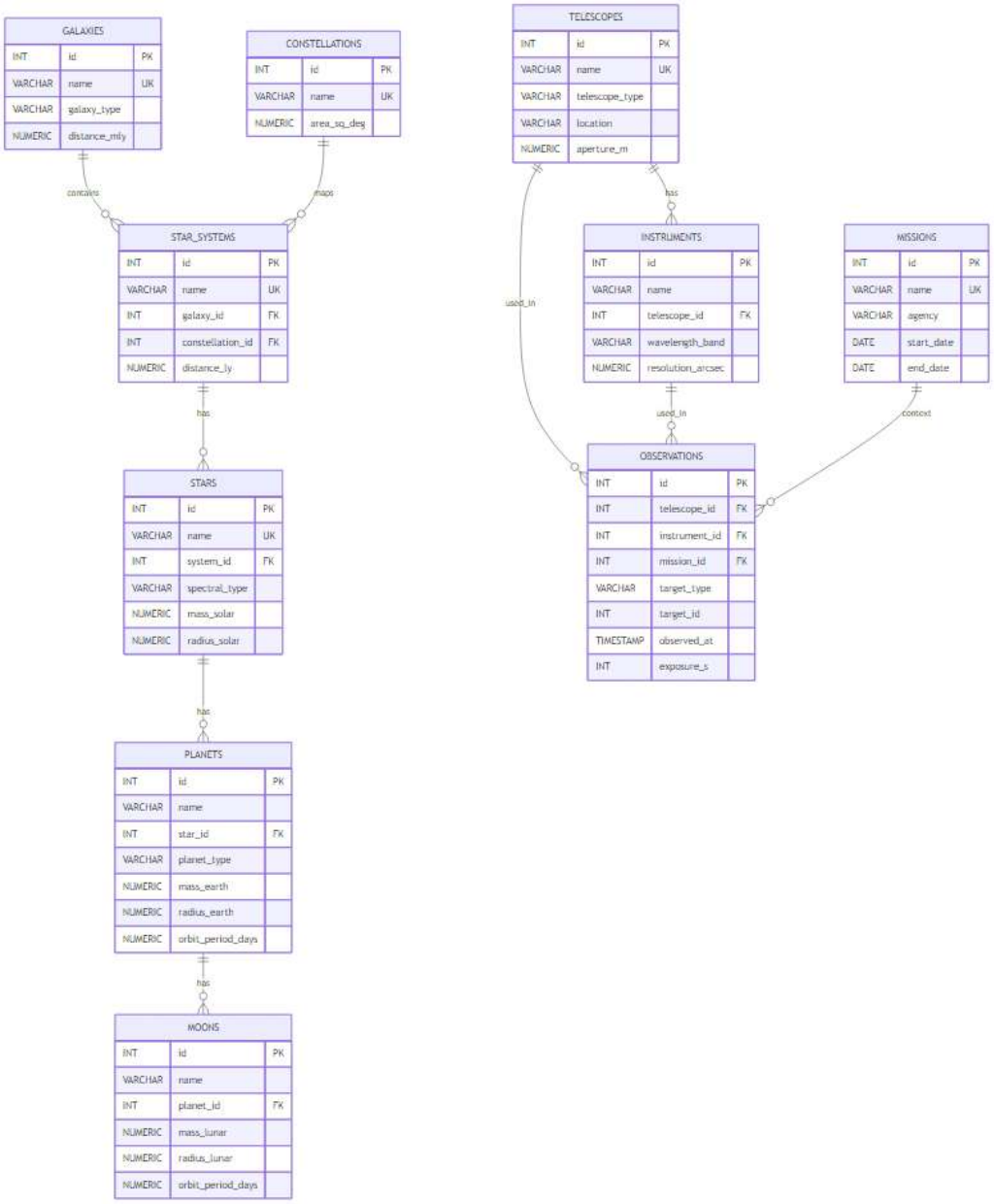
Podstawę hierarchii tworzą galaktyki, konstelacje, systemy gwiezdne, gwiazdy, planety i księżyce. Struktura ta pozwala badać zarówno proste odczyty, jak i zapytania wieloetapowe, które przechodzą przez kilka poziomów zależności.

W modelu relacyjnym zbiór został znormalizowany i oparty na kluczach głównych oraz obcych, co ułatwia utrzymanie integralności danych. W modelu dokumentowym te same informacje mogą być przechowywane jako odrębne kolekcje lub częściowo zagnieżdżane, w zależności od profilu zapytań. W modelu grafowym encje występują jako węzły, a zależności jako relacje, co upraszcza analizę ścieżek i powiązań.

Dobór domeny astronomii jest uzasadniony tym, że dobrze odwzorowuje różne rodzaje relacji i naturalnie nadaje się do mapowania na więcej niż jeden model danych. Jednocześnie jest to domena intuicyjna, dzięki czemu interpretacja wyników testów pozostaje czytelna zarówno technicznie, jak i merytorycznie.

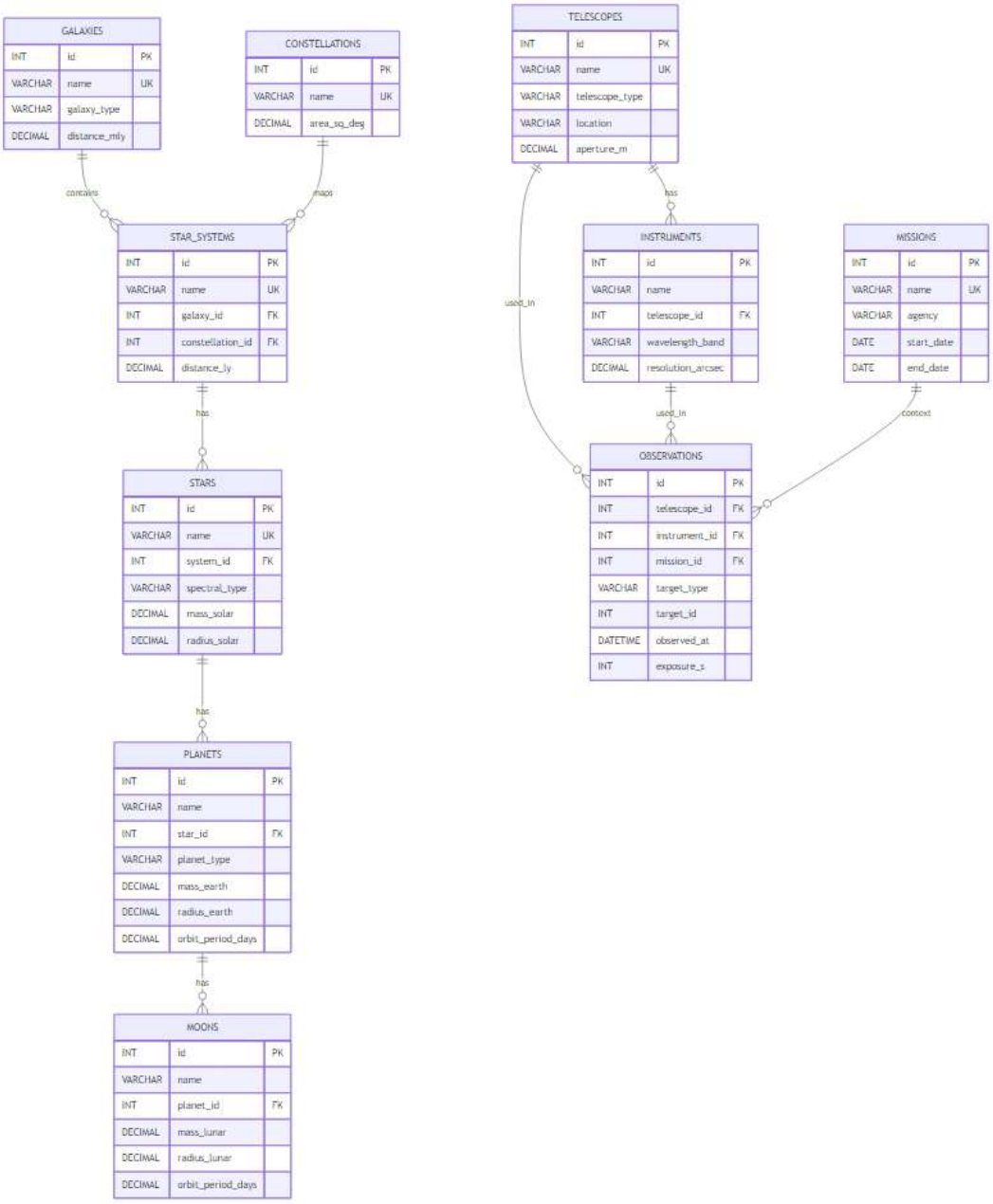
7. Opis architektury albumów

7.1 PostgreSQL



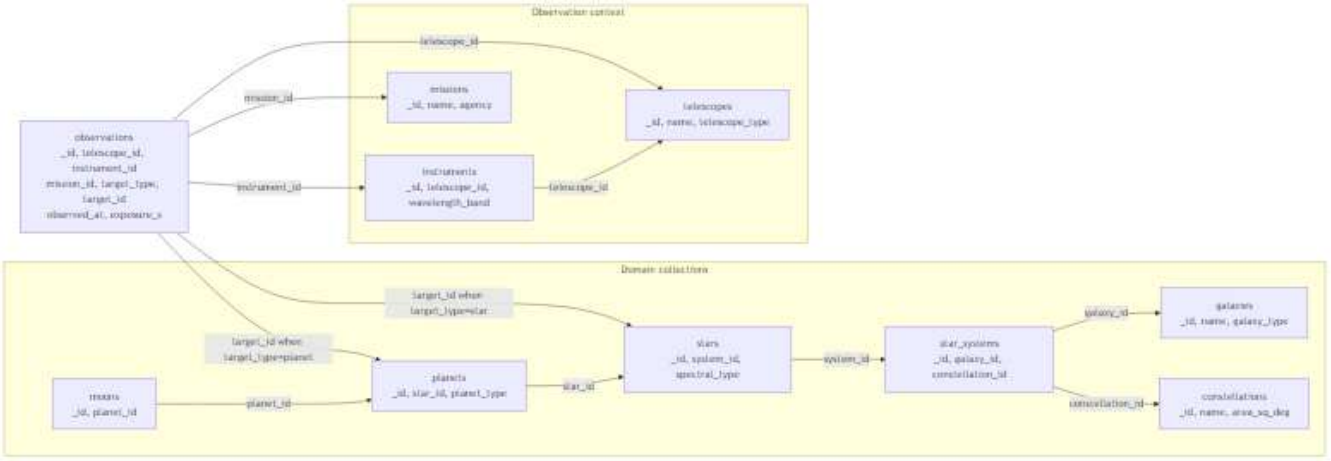
Architektura PostgreSQL została zaprojektowana jako klasyczny, znormalizowany model relacyjny, w którym dane są podzielone na dziesięć tabel odpowiadających głównym bytom domeny astronomicznej. Integralność danych jest zapewniona przez klucze główne i klucze obce, dzięki czemu zależności między galaktykami, systemami gwiezdnymi, gwiazdami, planetami i obserwacjami są jednoznaczne oraz łatwe do kontrolowania. Centralną rolę pełni tabela obserwacji, która łączy informacje o teleskopie, instrumencie, misji i obiekcie obserwowanym, co dobrze wspiera zapytania analityczne oraz porównania wydajnościowe.

7.2 MySQL



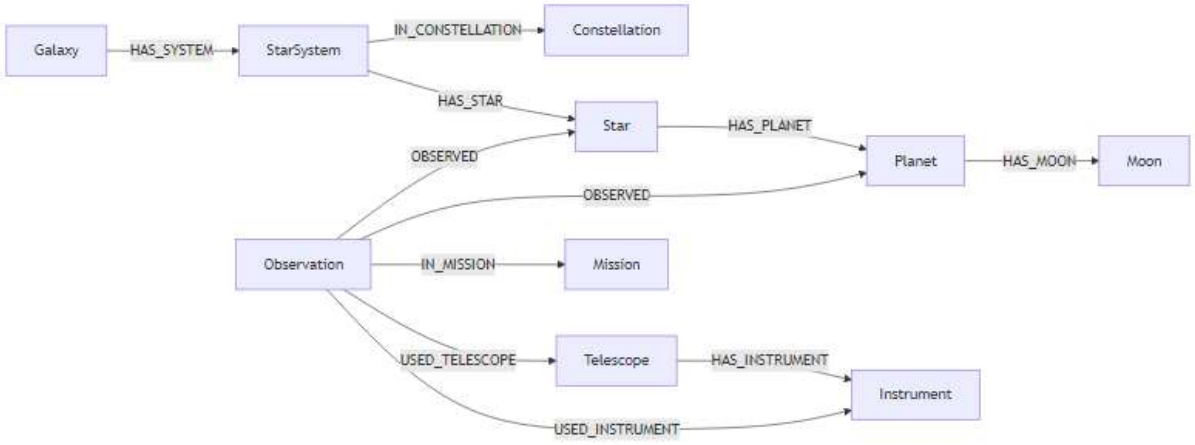
Architektura MySQL odwzorowuje ten sam model logiczny co PostgreSQL, ale korzysta z mechanizmów typowych dla silnika InnoDB i składni MySQL. Struktura relacji pozostaje spójna, dlatego system zachowuje wysoką przewidywalność zapytań i poprawną kontrolę powiązań pomiędzy danymi. Dzięki tej zgodności semantycznej MySQL stanowi dobre środowisko porównawcze wobec PostgreSQL, ponieważ różnice w wynikach dotyczą głównie charakterystyki silnika, a nie różnic w projekcie schematu.

7.3 MongoDB



Architektura MongoDB została przygotowana jako model dokumentowy z rozdzieleniem danych na kolekcje odpowiadające obszarom domenowym oraz kolekcję obserwacji jako główny element faktograficzny. Relacje między danymi są realizowane przez identyfikatory referencyjne, co daje większą elastyczność schematu i ułatwia ewolucję struktury dokumentów. Jednocześnie spójność relacji jest w większym stopniu odpowiedzialnością warstwy aplikacyjnej, ponieważ baza nie wymusza jej tak rygorystycznie jak klasyczne systemy relacyjne.

7.4 Neo4j



Architektura Neo4j została zbudowana jako model grafowy, w którym węzły reprezentują obiekty domenowe, a relacje opisują ich rzeczywiste powiązania semantyczne. Takie podejście szczególnie dobrze odwzorowuje zależności wielopoziomowe, na przykład od galaktyki przez system gwiezdny do planety i konkretnej obserwacji. Węzeł obserwacji pełni funkcję łącznika kontekstu naukowego i technicznego, a zapytania o ścieżki i powiązania są naturalne oraz czytelne. W praktyce ta architektura dobrze sprawdza się przy analizie relacji, choć wymaga świadomego strojenia zasobów pamięciowych w środowisku uruchomieniowym.

8. Przeprowadzone testy

Celem eksperymentu było porównanie wydajności czterech systemów bazodanowych dla jednolitego modelu danych astronomicznych. Zakres badań obejmował operacje CRUD wykonywane na danych o różnych wolumenach oraz analizę wpływu indeksowania na czas realizacji zapytań. Zestaw testowy składał się z 24 scenariuszy podzielonych na cztery grupy operacji.

Scenariusze CREATE:
W tej grupie badano czas tworzenia nowych rekordów, zarówno pojedynczych, jak i wsadowych.

- C1: dodanie pojedynczej misji.
- C2: dodanie pojedynczego teleskopu.
- C3: dodanie pojedynczego systemu gwiezdnego.
- C4: dodanie pojedynczej gwiazdy.
- C5: dodanie pojedynczej planety.
- C6: wsadowe dodanie 100 rekordów obserwacji.

Scenariusze READ:
W tej grupie badano odczyty punktowe, zakresowe, relacyjne i agregacyjne.

- R1: punktowe wyszukanie obserwacji po kluczu głównym.
- R2: odczyt zakresowy po czasie obserwacji z sortowaniem i limitem.
- R3: odczyt łączony pomiędzy obserwacjami i teleskopami.
- R4: agregacja średniego czasu ekspozycji w podziale na teleskop.
- R5: filtrowanie planet po typie i zakresie masy.
- R6: głęboki odczyt relacyjny od galaktyki do obiektów obserwowanych.

Scenariusze UPDATE:
W tej grupie badano aktualizacje pojedynczych rekordów i modyfikacje wsadowe.

- U1: aktualizacja pojedynczej obserwacji po identyfikatorze.
- U2: wsadowa aktualizacja obserwacji dla wybranej misji.
- U3: wsadowa aktualizacja rekordów planet.
- U4: wsadowa aktualizacja rekordów gwiazd.
- U5: wsadowa aktualizacja rekordów misji.
- U6: wsadowa aktualizacja rekordów instrumentów.

Scenariusze DELETE:
W tej grupie badano usuwanie punktowe i warunkowe.

- D1: usunięcie pojedynczej obserwacji po identyfikatorze.
- D2: usunięcie obserwacji z zakresu identyfikatorów.
- D3: usunięcie obserwacji powiązanych z wybraną misją.
- D4: usunięcie obserwacji dotyczących gwiazd.
- D5: usunięcie starszych rekordów obserwacji.
- D6: usunięcie obserwacji o niskim czasie ekspozycji.

Testy przeprowadzono dla trzech wolumenów danych: małego, średniego i dużego. Dla każdego wolumenu najpierw wykonywano etap ładowania danych, a następnie uruchamiano pełny zestaw 24 scenariuszy. Każdy scenariusz był wykonywany wielokrotnie (w kilku próbach), a wyniki zapisywano w postaci czasów wykonania w milisekundach.

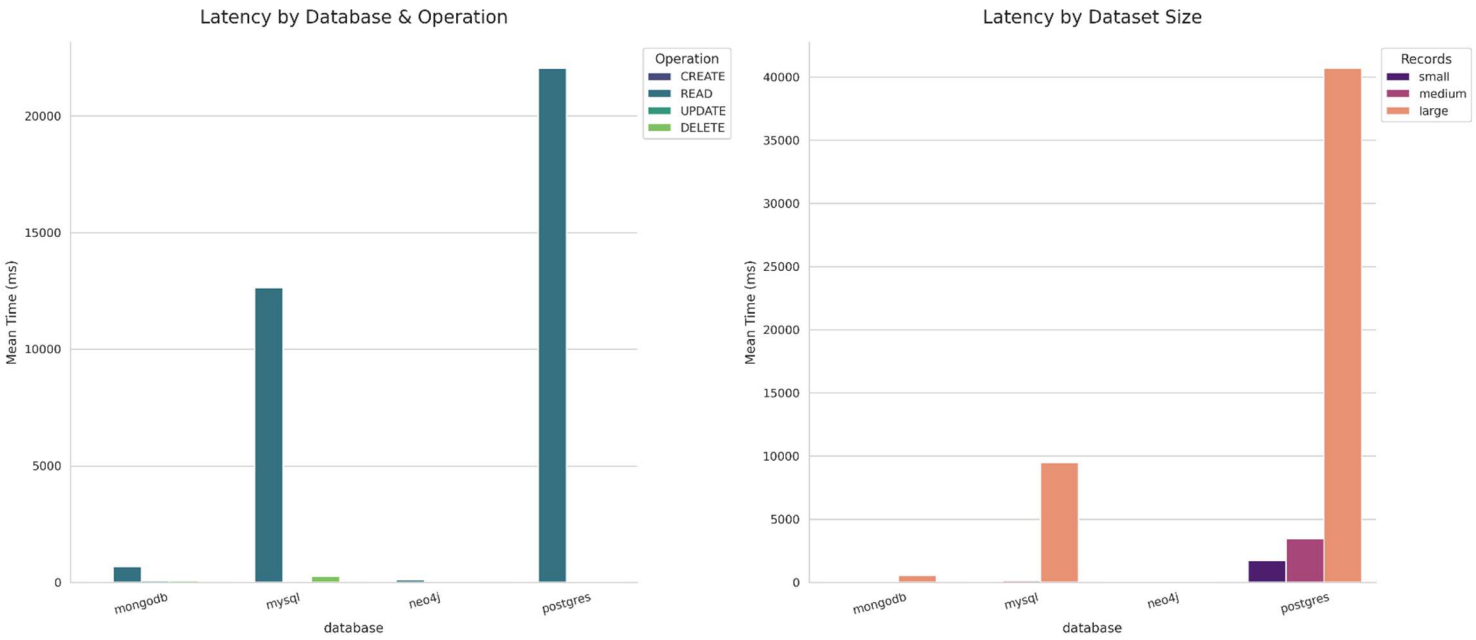
Metodologia obejmowała dwa tryby pracy: przed utworzeniem indeksów oraz po utworzeniu indeksów. W pierwszym trybie usuwano indeksy pomocnicze, aby wyznaczyć punkt odniesienia dla wydajności bazowej. W drugim trybie tworzono indeksy na polach najczęściej wykorzystywanych w odczytach i operacjach warunkowych, co pozwalało oszacować rzeczywisty wpływ indeksowania.

Aby zapewnić porównywalność pomiarów, każda próba scenariusza była realizowana w kontrolowanym kontekście wykonania, a wpływ zmian danych na kolejne przebiegi ograniczano przez izolację operacji i cofanie modyfikacji tam, gdzie było to technicznie możliwe. Dzięki temu wyniki między próbami i między systemami pozostawały spójne metodycznie.

Dodatkowo dla scenariuszy odczytowych rejestrowano plany wykonania zapytań (EXPLAIN/PROFILE), co umożliwiło interpretację różnic czasowych nie tylko na poziomie wartości liczbowych, ale również na poziomie strategii wykonania zastosowanej przez silnik bazy danych.

9. Analiza wyników:

- **Analiza skalowalności i latencji systemów bazodanowych**



Wykres przedstawia dwie perspektywy wydajności: po lewej stronie średni czas wykonania operacji w podziale na silnik bazy i typ operacji, a po prawej stronie średni czas wykonania w podziale na rozmiar danych dla każdej bazy. Innymi słowy, lewy panel odpowiada na pytanie „które operacje są najdroższe w danym systemie”, a prawy panel odpowiada na pytanie „jak bardzo rośnie koszt wraz ze skalą danych”.

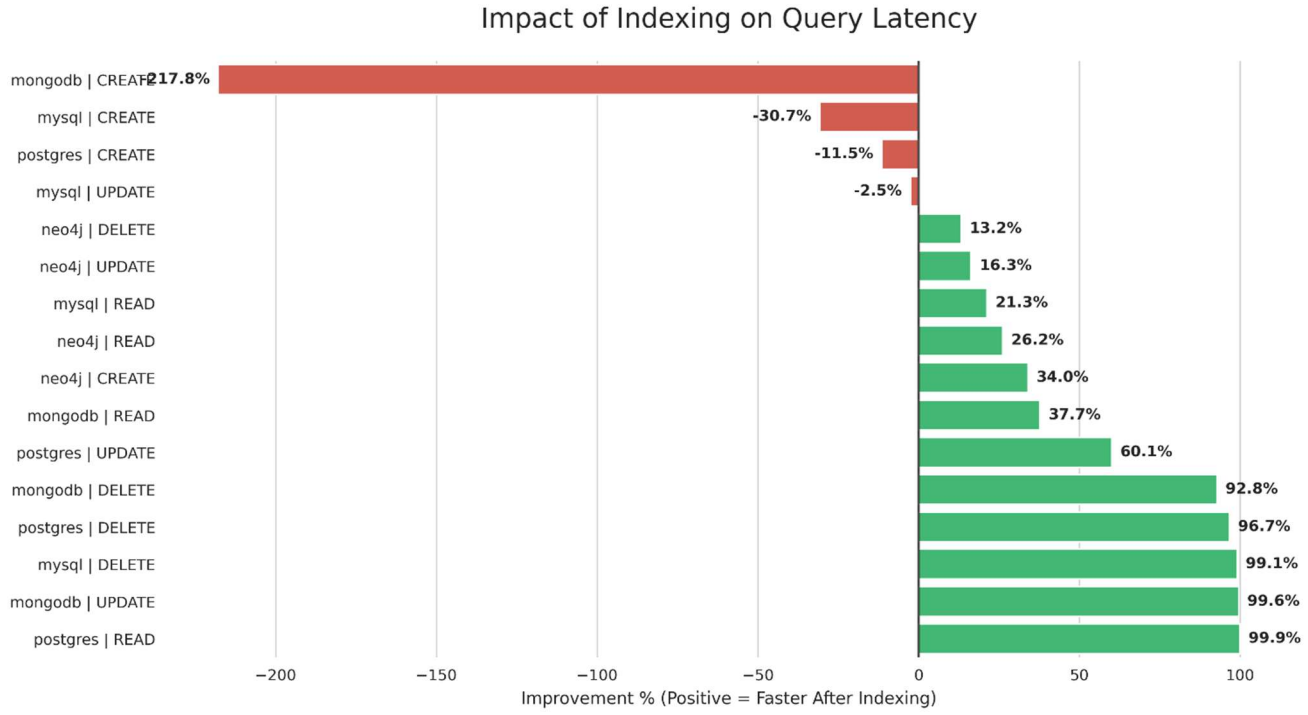
Z lewego panelu wynika przede wszystkim, że dominującym kosztem są operacje odczytu, zwłaszcza w PostgreSQL i MySQL. Słupki READ dla tych dwóch baz są o rząd wielkości większe niż słupki CREATE, UPDATE i DELETE, co oznacza, że to właśnie scenariusze odczytowe generują główne obciążenie całego benchmarku. To jest spójne z charakterem testów, ponieważ scenariusze READ obejmują nie tylko proste lookupy, ale też zapytania zakresowe, łączenia wielu encji i agregacje.

MongoDB na tym wykresie wypada wyraźnie lepiej od MySQL i PostgreSQL pod względem średniego czasu READ, ale nadal koszt odczytu jest tam zdecydowanie wyższy niż pozostałe operacje. Oznacza to, że również w modelu dokumentowym to odczyty są „wąskim gardłem”, tylko skala tego efektu jest mniejsza niż w bazach relacyjnych. Neo4j ma najniższe wartości niemal we wszystkich grupach operacji, co sugeruje bardzo dobrą wydajność w naszym zestawie testowym.

Prawy panel bardzo wyraźnie pokazuje wpływ wolumenu danych. Dla PostgreSQL wzrost jest silnie nieliniowy tzn. przejście od small do medium zwiększa czas zauważalnie, a przejście do large powoduje gwałtowny skok. To wskazuje, że dla dużych danych rośnie udział kosztownych planów wykonania, operacji I/O i ewentualnie sortowań lub skanów sekwencyjnych. Dla MySQL również widać duży wzrost przy large, choć mniej dramatyczny niż w PostgreSQL. MongoDB rośnie łagodniej, co sugeruje lepszą stabilność wraz ze wzrostem danych w ramach tego konkretnego zestawu zapytań. Neo4j pozostaje bardzo nisko również dla większych wolumenów na wykresie, co wygląda korzystnie.

Najważniejszy wniosek merytoryczny jest taki, że skalowanie odczytów jest kluczowym problemem relacyjnych silników w tym eksperymencie, a nie same operacje modyfikujące. W praktyce oznacza to, że optymalizacja powinna koncentrować się przede wszystkim na zapytaniach READ: doborze indeksów pod warunki WHERE i ORDER BY, ograniczaniu kosztownych joinów, denormalizacji wybranych fragmentów modelu dla najczęstszych raportów oraz kontroli planów wykonania dla scenariuszy R2–R6. Jeśli celem systemu produkcyjnego byłaby analityka na dużych wolumenach, to właśnie ta grupa zapytań będzie determinować odczuwaną wydajność.

- **Analiza efektu indeksowania na latencję zapytań**



Wykres pokazuje wpływ indeksowania na opóźnienie zapytań w rozbiciu na parę operacji, gdzie wartość dodatnia oznacza przyspieszenie po utworzeniu indeksów, a wartość ujemna oznacza pogorszenie. Jest to więc bezpośrednia analiza efektu „przed indeksami” vs „po indeksach”.

Na poziomie ogólnym wniosek jest bardzo wyraźny tzn. indeksy najmocniej pomagają operacjom odczytu i części operacji warunkowych, natomiast w niektórych przypadkach mogą pogarszać operacje zapisu, zwłaszcza CREATE. To jest zgodne z teorią działania indeksów, ponieważ indeks skraca czas wyszukiwania, ale jednocześnie zwiększa koszt utrzymania struktur podczas inserta i części update’ów.

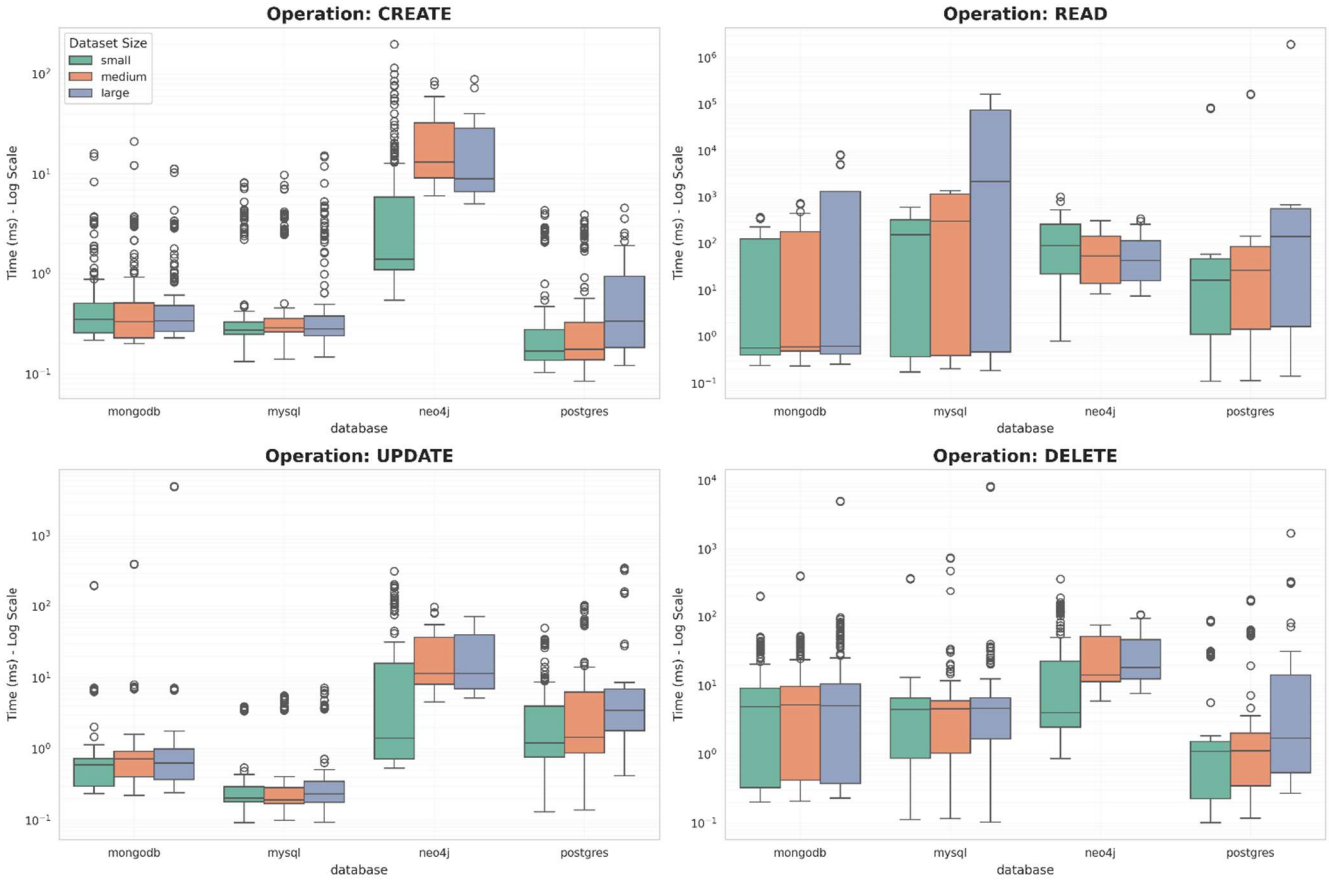
Najsilniejszy pozytywny efekt widać dla odczytów, szczególnie w PostgreSQL, a także wyraźnie w MongoDB i MySQL. Taki wynik oznacza, że zapytania z filtrami i sortowaniem zaczęły korzystać z indeksów zamiast pełnego skanowania danych. W praktyce to potwierdza, że kluczowe scenariusze READ zostały dobrze pokryte indeksami i to właśnie one były głównym źródłem zysków wydajnościowych.

Bardzo duże dodatnie wartości dla DELETE i części UPDATE sugerują, że usuwanie i modyfikacje wykonywane po warunkach również mocno skorzystały z szybszego odnajdywania rekordów. Innymi słowy, indeks nie tyle przyspiesza samo usunięcie rekordu, ile etap selekcji danych do usunięcia. To ważne, bo w raportach często błędnie interpretuje się takie wyniki jako szybszy zapis, podczas gdy faktycznie przyspiesza głównie wyszukiwanie docelowych rekordów.

Po stronie negatywnej najbardziej odstaje MongoDB dla CREATE, co oznacza istotne spowolnienie po indeksowaniu. Taki efekt jest logiczny, ponieważ każdy insert wymaga aktualizacji dodatkowych struktur indeksowych, a przy dużej liczbie insertów i wielu indeksach koszt ten może być dominujący. Ujemne wartości dla CREATE w MySQL i PostgreSQL pokazują ten sam mechanizm, tylko o słabszej skali.

Wyniki Neo4j są umiarkowanie dodatnie dla większości operacji, co wskazuje, że indeksy pomagają, ale efekt nie jest tak ekstremalny jak w relacyjnych odczytach. To zwykle oznacza, że część zapytań i tak korzystała z relacji/traversali efektywnie, a indeksy dołożyły poprawę głównie w punktach wejścia do grafu.

- **Analiza rozkładu latencjioperacji CRUD ezględem typu bazy i skali danych**



Powyższy wykres przedstawia rozkład czasów wykonania operacji CRUD dla trzech wolumenów danych. Zastosowano wykresy pudełkowe w skali logarytmicznej czasu, co umożliwia jednoczesną obserwację mediany, zmienności wyników oraz wartości odstających.

W każdym panelu mediana obrazuje typowy czas wykonania operacji, natomiast rozstęp międzykwartylowy wskazuje, jak bardzo wyniki różnią się między próbami. Wąsy oraz punkty poza pudełkiem reprezentują obserwacje skrajne, które są istotne z punktu widzenia stabilności działania systemu. Dzięki skali logarytmicznej możliwe jest uchwycenie różnic o kilka rzędów wielkości, których nie byłoby widać w skali liniowej.

Analiza panelu CREATE wskazuje, że operacje tworzenia rekordów są relatywnie stabilne dla części silników, jednak przy większych wolumenach danych rośnie zarówno mediana, jak i rozrzut wyników. Szczególnie widoczne jest to tam, gdzie utrzymanie dodatkowych struktur danych lub indeksów podnosi koszt zapisu. Oznacza to, że efektywność operacji tworzenia zależy nie tylko od samego insertu, ale także od obciążeń towarzyszących.

W panelu READ zaobserwowano największą zmienność wyników i najwyższe wartości odstające, zwłaszcza dla wolumenu large. Świadczy to o tym, że odczyt jest najbardziej wrażliwą grupą operacji w badanym obciążeniu. W praktyce oznacza to większą podatność na kosztowne plany zapytań, skanowanie dużych zakresów danych, sortowania i operacje łączenia. W konsekwencji to właśnie odczyty w największym stopniu determinują końcową wydajność całego systemu.

Panel UPDATE pokazuje umiarkowane mediany przy jednoczesnym wzroście rozrzutu wraz ze skalą danych. Taki rezultat sugeruje, że aktualizacje są zwykle przewidywalne dla małych wolumenów, natomiast przy większych zbiorach rośnie ryzyko wystąpienia opóźnień skrajnych. Wskazuje to na potrzebę optymalizacji warunków selekcji rekordów aktualizowanych oraz kontroli wpływu indeksów na koszt modyfikacji.

Panel DELETE również potwierdza zależność między wzrostem wolumenu a pogorszeniem stabilności czasów. Choć mediany mogą pozostawać relatywnie niskie dla mniejszych danych, przy wolumenie large pojawiają się wyraźne piki opóźnień. Wynika to z faktu, że koszt usuwania obejmuje nie tylko samo skasowanie rekordów, lecz także etap ich wyszukania i aktualizację struktur pomocniczych.

W ujęciu przekrojowym wykres potwierdza, że porównanie systemów bazodanowych wyłącznie na podstawie średniej wartości czasu jest niewystarczające. Kluczowe znaczenie ma analiza rozkładu, ponieważ to zmienność i wartości odstające decydują o przewidywalności działania systemu w warunkach rzeczywistych. W badaniu największym źródłem ryzyka wydajnościowego są operacje odczytu dla dużych wolumenów danych, natomiast operacje modyfikujące są zwykle bardziej stabilne, choć również wykazują okresowe odchylenia.

- **Analiza scenariuszowa wydajności baz danych na podstawie mapy cieplnej czasów operacji CRUD:**

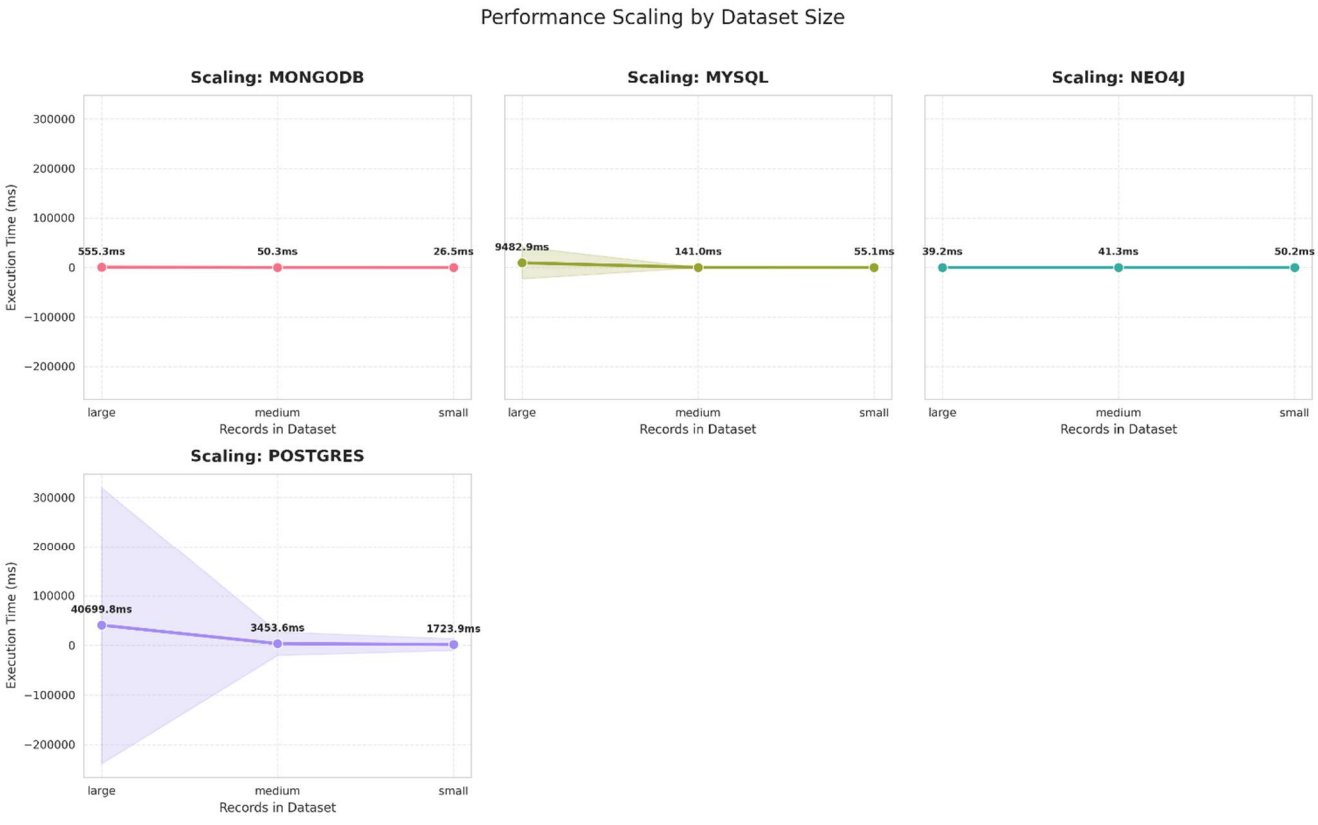


Przedstawiona mapa cieplna obrazuje czasy realizacji ustalonych scenariuszy testowych. Oś pozioma reprezentuje scenariusze testowe, natomiast oś pionowa odpowiada badanym systemom baz danych. Intensywność koloru odzwierciedla czas wykonania operacji w milisekundach, przy czym zastosowano skalę logarytmiczną, co umożliwia jednoczesną obserwację zarówno wartości niskich, jak i bardzo wysokich.

Analiza wykresu wskazuje, że największe różnice wydajnościowe występują w scenariuszach odczytowych, zwłaszcza dla zapytań bardziej złożonych (R3, R4, R6). Oznacza to, że to nie operacje zapisu, lecz operacje selekcji, agregacji i przetwarzania relacji mają dominujący wpływ na całkowity czas przetwarzania. Scenariusze CREATE, a także znacząca część UPDATE i DELETE, wykazują w większości przypadków niższe i bardziej stabilne czasy wykonania co jest zgodne z pozostałymi przeprowadzonymi analizami.

Na podstawie mapy cieplnej można stwierdzić, że wydajność badanych systemów zależy przede wszystkim od charakteru zapytania, a nie od samej kategorii bazy danych. Kluczowym wnioskiem praktycznym jest konieczność optymalizacji scenariuszy odczytowych, ponieważ to one generują największy koszt i odpowiadają za główne różnice pomiędzy silnikami. W ujęciu metodologicznym mapa cieplna potwierdza, że ocena wydajności powinna być prowadzona scenariuszowo.

- **Analiza skalowalności wydajności baz danych względem wolumenu danych**



Przedstawiony wykres obrazuje zależność średniego czasu wykonania operacji od rozmiaru zbioru danych. Każdy panel prezentuje osobny silnik, co pozwala bezpośrednio porównać jego zachowanie przy wzroście wolumenu danych i ocenić charakter skalowania.

Dla MongoDB odnotowano trend malejący wraz ze wzrostem rozmiaru etykietowanego od prawej do lewej strony wykresu, przy wartościach około 26.5 ms, 50.3 ms i 555.3 ms dla odpowiadających punktów. Oznacza to, że dla największego zbioru danych czas wykonania rośnie, jednak pozostaje istotnie niższy niż w części pozostałych silników. Wskazuje to na relatywnie dobrą stabilność wydajnościową, choć widoczny jest wpływ skali danych.

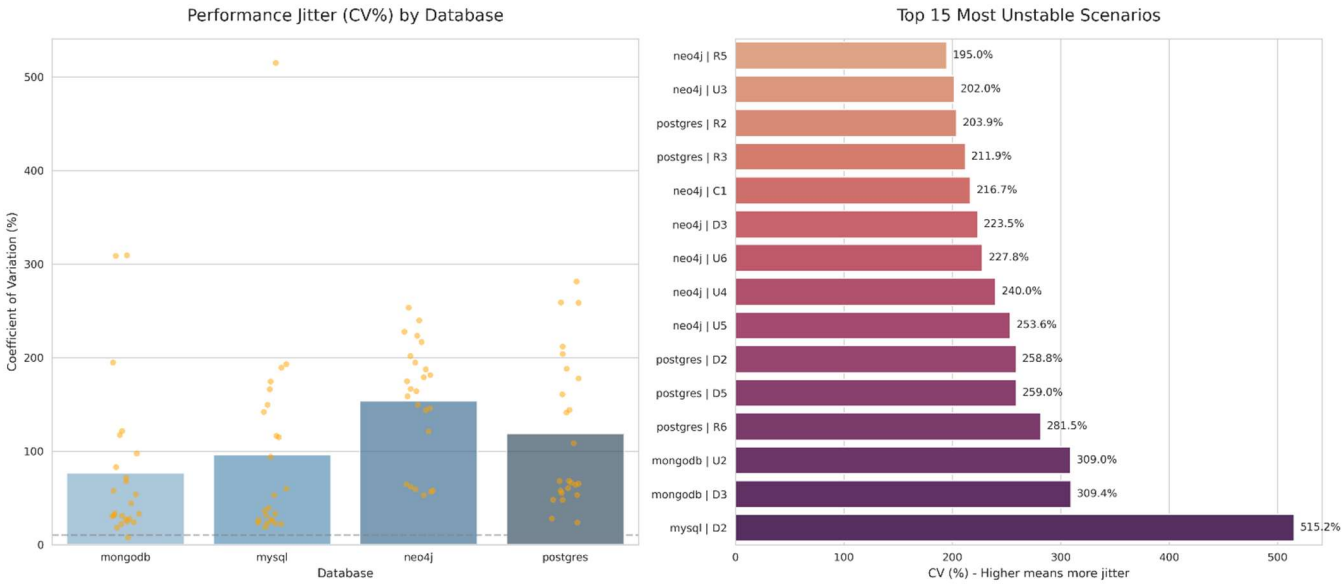
W przypadku MySQL obserwowany jest silniejszy efekt degradacji wydajności przy dużym wolumenie danych: około 55.1 ms dla small, 141.0 ms dla medium i aż 9482.9 ms dla large. Taki skok sugeruje, że przy największym rozmiarze danych pojawiają się kosztowne plany zapytań lub operacje wejścia/wyjścia, które dominują czas wykonania. Oznacza to, że skalowalność MySQL w badanym obciążeniu jest ograniczona i silnie zależna od charakteru zapytań.

Neo4j prezentuje najbardziej wyrównany profil spośród pokazanych paneli. Czasy wykonania są zbliżone dla wszystkich rozmiarów danych, co wskazuje na wysoką stabilność i niską wrażliwość na zmianę wolumenu w analizowanym zakresie. Wynik ten sugeruje dobre dopasowanie modelu grafowego do badanego typu operacji oraz brak gwałtownych kosztów narastających wraz ze skalą danych.

PostgreSQL wykazuje najwyższe wartości bezwzględne i najsilniejszą degradację wraz ze wzrostem rozmiaru danych: około 1723.9 ms dla small, 3453.6 ms dla medium i 40699.8 ms dla large. Oznacza to, że przy największym wolumenie danych koszt wykonania rośnie bardzo intensywnie i staje się głównym ograniczeniem wydajnościowym. Jednocześnie szeroki obszar cieniowania w panelu PostgreSQL wskazuje na dużą zmienność wyników, a więc mniejszą przewidywalność czasu odpowiedzi.

Porównanie między silnikami prowadzi do wniosku, że najlepszą stabilność skalowania w badanym zakresie prezentuje Neo4j, natomiast największą wrażliwość na wzrost danych wykazuje PostgreSQL, a następnie MySQL. MongoDB zajmuje pozycję pośrednią: skaluje się zauważalnie lepiej od silników relacyjnych w tym eksperymencie, ale gorzej od Neo4j pod względem stabilności trendu.

- **Analiza stabilności wydajności baz danych na podstawie współczynnika zmienności CV**



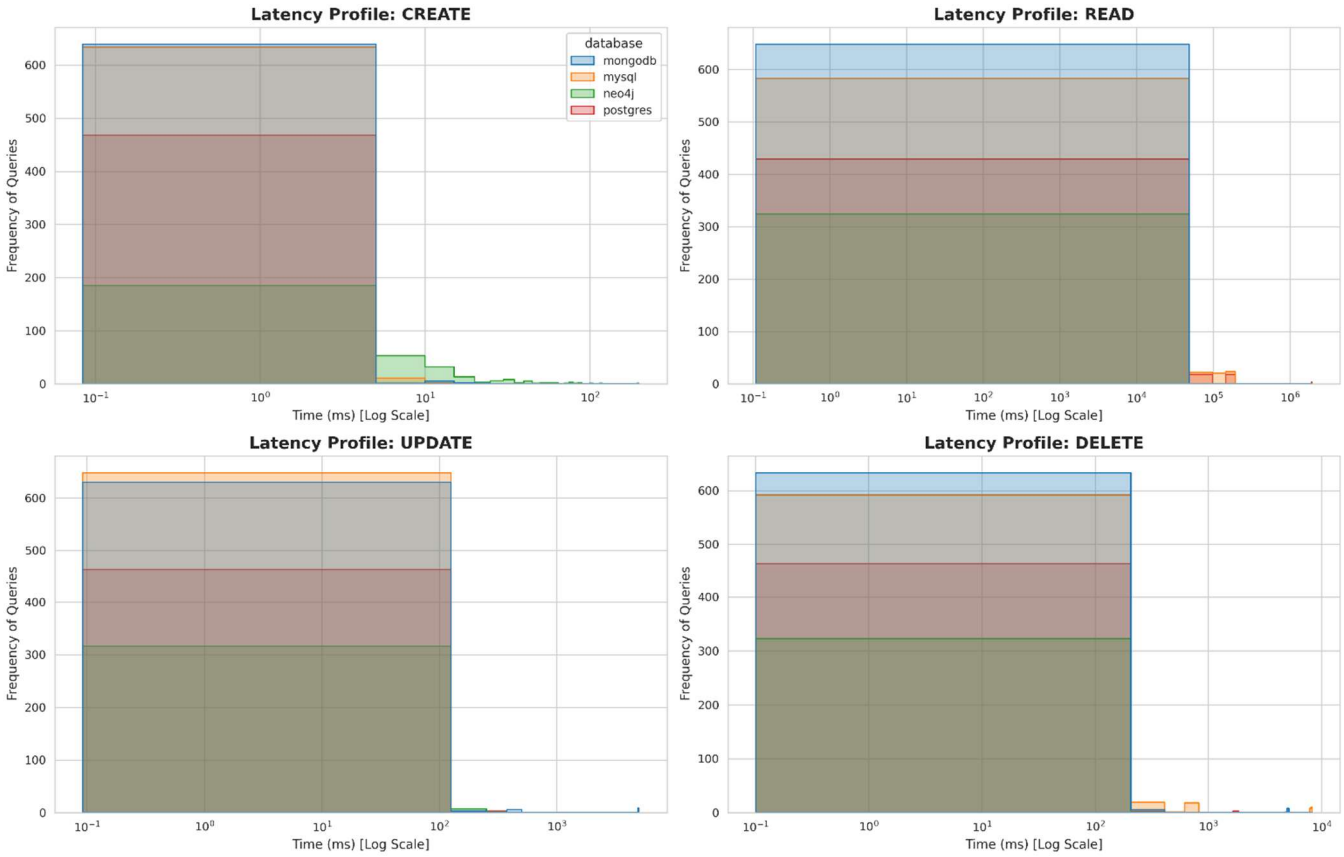
Przedstawiony zestaw wykresów służy do oceny stabilności czasów wykonania scenariuszy testowych w badanych silnikach bazodanowych. Jako miarę niestabilności przyjęto współczynnik zmienności (CV%), czyli relację odchylenia standardowego do średniej. Im wyższa wartość CV, tym większy jitter wydajnościowy i mniejsza przewidywalność czasu odpowiedzi.

Lewy panel pokazuje rozkład wartości CV dla wszystkich scenariuszy w podziale na bazy danych. Widać, że najniższy typowy poziom zmienności występuje dla MongoDB, natomiast najwyższy dla Neo4j i PostgreSQL. MySQL prezentuje poziom pośredni, ale z obecnością bardzo wysokich wartości odstających. Oznacza to, że ocena wydajności nie może ograniczać się wyłącznie do średnich czasów, ponieważ część scenariuszy charakteryzuje się dużą niestabilnością pomiędzy kolejnymi próbami.

Prawy panel przedstawia 15 najbardziej niestabilnych scenariuszy. Najwyższą wartość uzyskał scenariusz mysql (D2), co wskazuje na bardzo silne wahania czasów usuwania. Kolejne wysokie pozycje zajmują mongodb (D3) i mongodb (U2), a także kilka scenariuszy PostgreSQL (R6, D5, D2) oraz Neo4j (U5, U4, U6, D3, C1, R5). Taki rozkład potwierdza, że niestabilność ma charakter scenariuszowy i zależy od typu operacji, a nie tylko od wybranej technologii.

Szczególnie istotny jest fakt, że wśród najbardziej niestabilnych przypadków dominują operacje DELETE, UPDATE i część złożonych READ. Wskazuje to, że jitter rośnie tam, gdzie silnik musi wykonać więcej operacji warunkowych, selekcji rekordów lub aktualizacji struktur pomocniczych. W praktyce oznacza to ryzyko nieprzewidywalnych czasów odpowiedzi nawet wtedy, gdy średni czas dla danej operacji wygląda akceptowalnie.

- **Analiza profili latencji operacji CRUD z uwzględnieniem rozkładów czasów odpowiedzi:**



Przedstawiony wykres prezentuje profile latencji dla operacji CRUD w podziale na badane silniki bazodanowe. Każdy panel przedstawia histogram rozkładu czasów odpowiedzi, a oś czasu została zaprezentowana w skali logarytmicznej.

Najważniejszą obserwacją jest silna asymetria rozkładów, czyli dominacja krótkich czasów odpowiedzi przy jednoczesnej obecności długich ogonów. Oznacza to, że większość zapytań wykonywana jest szybko, jednak okresowo pojawiają się pojedyncze lub nieliczne przebiegi o bardzo wysokich opóźnieniach.

W panelu CREATE rozkłady są relatywnie najbardziej skupione po lewej stronie osi, co wskazuje na przewagę niskich latencji dla operacji tworzenia danych. Jednocześnie widoczne są pojedyncze odchylenia dla części silników, co może wynikać z kosztu utrzymania indeksów lub chwilowych opóźnień I/O. W ujęciu ogólnym operacje CREATE cechują się wyższą przewidywalnością niż odczyty.

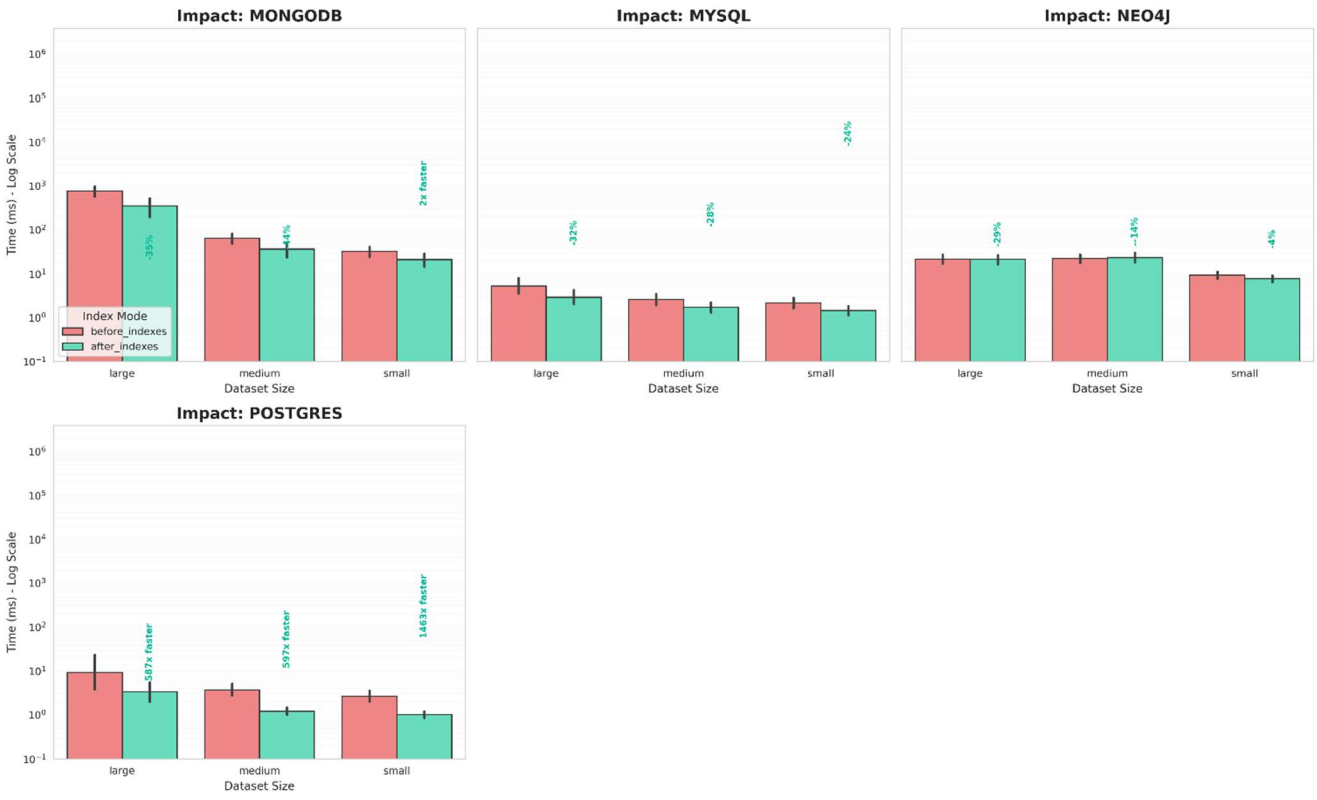
Panel READ przedstawia najbardziej rozciągnięte rozkłady i najdłuższe ogony, sięgające bardzo wysokich wartości. To potwierdza, że odczyty są najbardziej wrażliwą wydajnościowo klasą operacji w badanym środowisku. W praktyce oznacza to, że zapytania generują największe ryzyko występowania skrajnych opóźnień i to one powinny być priorytetem optymalizacyjnym.

W panelu UPDATE widoczny jest profil pośredni tzn. większość wyników znajduje się w niskim zakresie, ale pojawiają się również wartości znacznie wyższe, wskazujące na niestabilność dla wybranych scenariuszy. Może to wynikać z różnego kosztu selekcji rekordów do aktualizacji oraz wpływu indeksowania na modyfikację danych.

Panel DELETE pokazuje podobny charakter do UPDATE, jednak z wyraźnymi ogonami w wyższych zakresach latencji dla części przypadków. Oznacza to, że usuwanie jest zwykle szybkie, ale scenariusze warunkowe lub obejmujące większe zakresy danych mogą powodować skokowe pogorszenie czasu wykonania.

W przekroju wszystkich paneli można stwierdzić, że różnice między silnikami bazodanowymi nie ograniczają się do mediany czasu odpowiedzi, lecz przede wszystkim dotyczą kształtu całego rozkładu. Dwa systemy o podobnej typowej latencji mogą istotnie różnić się stabilnością, jeśli jeden z nich ma dłuższy ogon rozkładu i więcej epizodów bardzo wolnych odpowiedzi.

- **Wpływ indeksowania na skalowalność wydajności baz danych**



Przedstawiony wykres porównuje czasy wykonania zapytań przed i po utworzeniu indeksów dla trzech wolumenów danych. Każdy panel odpowiada jednej bazie, a słupki zestawiają dwa tryby testowe: before_indexes i after_indexes. Oś czasu jest zaprezentowana w skali logarytmicznej.

W MongoDB po zastosowaniu indeksów widoczna jest poprawa wydajności we wszystkich wolumenach, przy czym efekt jest umiarkowany dla large i medium, a wyraźniejszy dla small. Oznacza to, że indeksy skutecznie redukują koszt wyszukiwania danych, ale skala zysku zależy od charakteru zapytań i rozmiaru zbioru.

W MySQL indeksowanie również przynosi konsekwentną poprawę, rzędu około 24–32% w zależności od wolumenu danych. Trend jest stabilny i sugeruje, że indeksy zostały dobrze dopasowane do najczęściej wykonywanych warunków filtrowania i odczytu. W praktyce wskazuje to na przewidywalny, dodatni efekt optymalizacji niezależnie od wielkości danych.

W Neo4j poprawa po indeksowaniu jest widoczna, ale mniejsza niż w MySQL i MongoDB. Dla large i medium zysk wynosi kilkanaście do około 30%, natomiast dla small jest marginalny. Taki wynik jest typowy dla bazy grafowej, gdzie część zapytań i tak korzysta głównie z traversali relacji, a indeksy pomagają przede wszystkim na etapie odnajdywania punktu wejścia do grafu.

Najsilniejszy efekt obserwowany jest w PostgreSQL, gdzie po indeksowaniu czasy wykonania spadają bardzo wyraźnie, a adnotacje wskazują nawet wielokrotne przyspieszenia. Taki rezultat sugeruje, że przed indeksowaniem część zapytań wykonywała kosztowne skanowania lub nieefektywne plany wykonania, które po dodaniu indeksów zostały radykalnie zoptymalizowane. Jednocześnie tak duże mnożniki należy interpretować ostrożnie, ponieważ mogą wynikać z obecności pojedynczych scenariuszy o ekstremalnie wysokim czasie bazowym.

10. Wnioski:

Przeprowadzone badania potwierdziły, że wydajność systemu bazodanowego zależy przede wszystkim od charakteru zapytań, a nie wyłącznie od klasy zastosowanego silnika. Największe różnice między technologiami ujawniły się w scenariuszach czytania danych, szczególnie tych obejmujących operacje złożone, agregacje i wieloetapowe przejścia po relacjach.

W analizie średnich czasów wykonania największym obciążeniem okazały się operacje READ, podczas gdy CREATE, UPDATE i DELETE były zwykle mniej kosztowne. Oznacza to, że w praktyce produkcyjnej kluczowym obszarem optymalizacji powinny być zapytania odczytowe, a nie same operacje zapisu.

Analiza wpływu indeksowania wykazała dodatni efekt dla większości scenariuszy odczytowych i warunkowych. Jednocześnie zaobserwowano, że indeksy mogą pogarszać część operacji zapisu, zwłaszcza CREATE, co jest

zgodne z teorią utrzymania struktur indeksowych. Wniosek praktyczny jest taki, że strategia indeksowania musi wynikać z realnego profilu obciążenia.

Wyniki skalowalności pokazały, że wraz ze wzrostem wolumenu danych rośnie nie tylko średni czas odpowiedzi, ale także ryzyko występowania opóźnień skrajnych. Najbardziej wrażliwe na wzrost danych były scenariusze relacyjne o dużej złożoności. Jednocześnie należy podkreślić, że wyniki Neo4j były liczone dla praktycznych wolumenów 1k/5k/15k, co ogranicza bezpośrednio porównywalność z pozostałymi silnikami.

Analiza stabilności CV oraz percentyli latencji potwierdziła, że ocena wydajności oparta wyłącznie na średniej jest niewystarczająca. W kilku przypadkach mediany pozostawały akceptowalne, ale ogon rozkładu wskazywał ryzyko bardzo wolnych odpowiedzi. Z perspektywy jakości usług i SLA to właśnie zachowanie ogona rozkładu ma kluczowe znaczenie.

W przekrojowym ujęciu eksperyment wykazał, że nie istnieje jedna najlepsza baza dla wszystkich przypadków użycia. Silniki relacyjne oferują wysoką dojrzałość transakcyjną i spójność, ale wymagają starannej optymalizacji złożonych odczytów. MongoDB zapewnia elastyczność modelu i dobre wyniki dla prostszych operacji dokumentowych, natomiast Neo4j jest szczególnie efektywny tam, gdzie rdzeniem problemu są relacje i zapytania grafowe.